Strongly Entangling Neural Network: Quantum-Classical Hybrid Model for Quantum Natural Language Processing



J. Ismael Díaz-Ortiz, Axel Villanueva, and Francisco Delgado

Abstract One of the most used techniques to improve a Machine Learning model is to gather more data. An interesting field in Machine Learning is Sequence Modelling, having Natural Language Processing as the peak of the field. The capabilities of Quantum Computing have been growing recently entering the novel field of Quantum Machine Learning. In this paper, we propose a Quantum Natural Language Processing classification model named Strongly Entangling Neural Network. This model leverages the quantum advantage to imitate part of the behavior of a Recurrent Neural Network to process text data into the circuit and perform the classification task. This is accomplished by representing our data in a quantum circuit that relies heavily on the entanglement property of qubits. The results of our model have very favorable metrics, particularly obtaining a 97.70% of accuracy.

Keywords Quantum machine learning · Quantum natural processing · Quantum information

1 Introduction

Classical Machine Learning (CML) algorithms have proven to be powerful at finding the hidden patterns underneath large amounts of data. Nevertheless, the time and hardware requirements needed to train complex state-of-the-art models have been increasing in recent years. Together, the complexity, abstractness of the data, and desired tasks have also increased. With the rise in popularity and capabilities of quantum technologies and their counterintuitive algorithms of Quantum Machine Learning (QML) when compared with classical computing, it can be expected that QML algorithms running on quantum computers can outperform CML algorithms for classical computers [1]. A field recently growing at a rapid pace within CML is Natural Language Processing (NLP). The motivation behind NLP is to provide a computer with the ability to analyze, process, and find meaning inside large amounts

J. I. Díaz-Ortiz · A. Villanueva · F. Delgado (✉)
School of Engineering and Sciences, Tecnologico de Monterrey, Monterrey, Mexico
e-mail: fdelgado@tec.mx

© The Author(s), under exclusive license to Springer Nature Switzerland AG 2024

D. Vlachos (ed.), *Mathematical Modeling in Physical Sciences*, Springer Proceedings in Mathematics & Statistics 446, https://doi.org/10.1007/978-3-031-52965-8_40

of Natural Language Data (NLD). Once abstracted from the human perspective about how language is seen, the phrases or sentences can be understood just as a sequence of data, being then NLP a problem for Sequence Modelling (SM) [2].

In classical computing there exist several NLP algorithms capable of extracting the desired information from the text depending on the desired task: classification, text segmentation, Named Entity Recognition, etc. [3–5]. In the field of Deep Learning, the most popular models are based on the use of Convolutional Neural Networks, Recurrent Neural Networks (RNN) [6] and Transformers/Attention-based models [7, 8]. Some state-of-the-art models in NLP using these previous base architectures are Generative Pre-trained Transformer [9], Bidirectional Encoder Representations from Transformers [10], and Hugging Face's BLOOM [11]. The performance and possibilities given by QML algorithms opened a new field for NLP: Quantum Natural Language Processing (QNLP) based on the modeling of language sequences taking the advantage of quantum systems, which is the main topic of this research centered particularly on QNLP for classification tasks. In Fig. 1a it can be seen in better detail

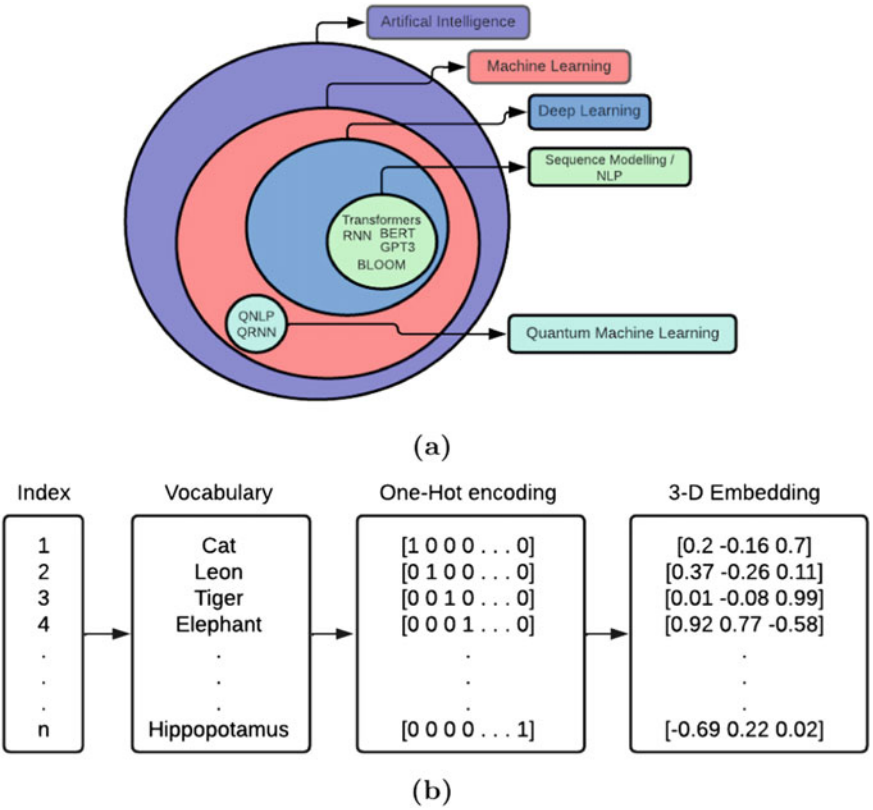


Fig. 1 **a** Interactions inside Artificial Intelligence with Deep Learning, Machine Learning, and Quantum Machine Learning. **b** One-hot encoding and Word embedding are illustrated. Closer words under certain criteria exhibit characteristic measures of similitude (inner product, distance measure, or another norm) among the corresponding vectors

how CML and QML are a subcategory derived from Artificial Intelligence, and how the different fields and techniques are categorized in general.

In this paper is proposed an architecture for SM made specifically for QNLP: *Strongly Entangling Neural Network*. It is based on the use of Variational Quantum Circuits (VQC) from which are generated circuit neurons to mimic the general structure of an RNN, without some explicit connections. The skeleton of the model is based on the Circuit-Centric Quantum classifier proposed in [12]. The structure of the article is as follows. The Fundamentals section presents the general theory and background regarding the model explained in deeper detail. Their corresponding subsections will work as building blocks for the proposed model. Then, the model is explained in detail in the section Strongly Entangling Neural Network. Experiment outcomes are shown in the Results section. Finally, conclusions and future work are presented in the last section.

2 Fundamentals

The aim of this section is to describe the different elements and techniques used, as well as the mathematical foundations related to Quantum information and Quantum processing underlying our proposed model. Also, the subsections correspond to the elements used in the final model being presented in the order that they appear: Word Embedding (text to vector representation), Quantum Embedding (classical to quantum data), VQC (Quantum circuit with trainable parameters), Measurements and Loss function for the deployment and optimization of the model.

2.1 Words Embedding

When talking about CML, it is needed to understand that a computer is not capable of understanding anything but numbers. Depending on the desired task for the NLP model, the architecture may vary considering the inputs and outputs. For this research, it is sought that computers understand and perform classification. The inputs are a sequence of data in the form of text, which should be transformed into a numeric representation. This process is known as Text Vectorization (TV) [13].

One way of doing TV is to generate a one-hot-encoding for the whole vocabulary. That is, to generate a vector representation considering a vocabulary of length n, then obtain a vector representation for each word with the same length n. There, just one value in the vector is 1, otherwise, 0 [13]. The problem with this representation is that the size of the vector increases directly with the size of the vocabulary. It represents a memory consuming, not properly convenient for efficient calculations. In addition, there is no relation between the words in the vocabulary.

Despite this problem, another way to generate the TV is Word Embedding (WE). It consists of a dense vector representation of n dimensions (chosen by the practitioner

depending on the data set), with floating-point values. Here, the values in the vector for every word are trainable parameters, so similar words (according to their meaning and/or context), once trained, have a similar or near vector representation. Figure 1b illustrates the one-hot-encoding and WE. It shows the transition from a set of words into a set of quantum states during the classical and quantum encoding (3D as an instance). Considering the benefits, WE was chosen as the TV algorithm for the proposed model.

2.2 *Encoding Layer and Angle Embedding*

The output of WE is still in the form of classical information, which naturally does not fit into a quantum circuit by itself. Therefore, an extra step called Quantum Embedding (QE) is needed. There, classical data is encoded into a quantum circuit. There are several methods of QE, some most used are: Amplitude Embedding, Basis Embedding, and Displacement Embedding, among others [14].

The method selected for the current model was Angle Embedding. Here, n features are encoded as the angles of rotations applied to n qubits after they are initialized in the state $|0\rangle^{\otimes n}$. X, Y or Z rotations can be used. A circuit example using $R_x(x_i)_i$ rotations on each qubit i is shown in Fig. 2. By introducing the classical information into the circuit in the form of rotations angles, the qubits initialized as $|0\rangle^{\otimes n}$ are now representatives of classical data as well as quantum data. In (1), it is shown how the classical data is encoded and introduced to the quantum circuit using the corresponding operation, considering an array of input features $X = [x_1, \ldots, x_n]$, where $S(X) = \bigotimes_{i=1}^{n} R_x(x_i)_i$, $x_i \in [-\pi, \pi]$) will describe the whole encoding block and $|\psi(x)\rangle$ is the state of the whole circuit after the encoding layer or encoding block. Then, it reads:

$$\underset{\text{representation}}{\text{Word binary}} \rightarrow [0, 1, \ldots, 0, 0] \rightarrow [x_1, \ldots, x_n] \rightarrow |\psi(x)\rangle = S(X)|0\rangle^{\otimes n} \quad (1)$$

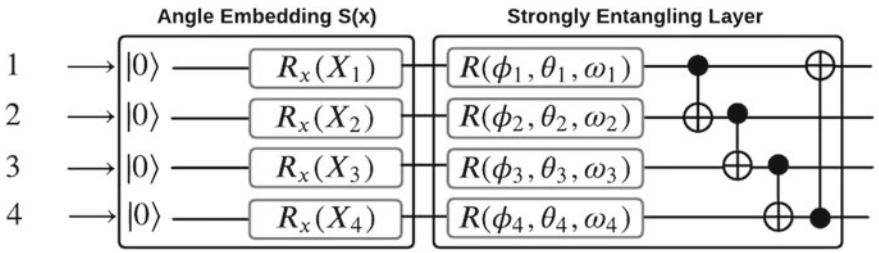


Fig. 2 EB $S(X)$ for the angle embedding and the Strongly Entangling VQC. This whole circuit corresponds to the Circuit Neuron used in the model

2.3 Variational Quantum Circuits and Ansatz

Analogous to how CML works, particularly in the field of Deep Learning, QML makes use of VQC, which as the name mentions, contains Variational parameters which are optimized during training depending on the architecture and task. With the use of VQC, a QML model can be described as follows (2). By considering a trainable set of parameters θ and input features X, $f(X, \theta)$ is the expectation value of some observable in the circuit while M is the observable. Then, $U(X, \theta)$ is VQC whose output depends on the input X and the trainable parameters set θ. $|\mathbf{0}\rangle$ corresponds to the initial state (there, $|\mathbf{0}\rangle = |\mathbf{0}\rangle^{\otimes \mathbf{n}}$). This equation considers that the encoding block described in the previous section is part of the VQC, $U(X, \theta)$:

$$f(X, \theta) = \langle \mathbf{0} | S(X)^{\dagger} U^{\dagger}(X, \theta) M U(X, \theta) S(X) | \mathbf{0} \rangle \tag{2}$$

There, the encoding layer will be referred as the Encoding Block (EB) $S(X)$. While the VQC having all trainable parameters doing the processing to get the proper output will be referred as the Training Block (TB) $U(\theta)$. Then, we define the initial state $|\psi(x)\rangle = S(X)|\mathbf{0}\rangle$ for the QML model after applying the EB.

$U(\theta)$ can be conformed by different layers depending on the whole circuit depth. In VQC, commonly several layers are used, each one conformed similarly but including its own trainable parameters. Thus, $U(\theta)$ can be described considering L layers as:

$$U(\theta) = U_L(\theta_L) \ldots U_2(\theta_2) U_1(\theta_1) \tag{3}$$

For this paper, the basis VQC used as a neuron for the whole model will only have one layer conformed by single qubit rotations $R(\phi, \theta, \omega)$ and 2-qubit unitary gates called *imprimitives*, which are not local gates, but controlled gates referred as entangling gates (as an instance, a controlled-NOT gate $C^a NOT_b$). As mentioned, in general, the single-qubit rotation depends on 3 parameters, which are the trainable parameters for that specific gate. The 3 parameter rotation is the product of other Pauli rotation gates. It is (4):

$$R(\phi, \theta, \omega) = R_z(\omega) R_y(\theta) R_z(\phi) = \begin{pmatrix} e^{-\frac{i}{2}(\phi+\omega)} \cos(\frac{\theta}{2}) & -e^{+\frac{i}{2}(\phi-\omega)} \sin(\frac{\theta}{2}) \\ e^{-\frac{i}{2}(\phi-\omega)} \sin(\frac{\theta}{2}) & e^{+\frac{i}{2}(\phi+\omega)} \cos(\frac{\theta}{2}) \end{pmatrix} \tag{4}$$

while, the controlled gate has the form (G^0 is the identity):

$$C^i G_j |i\rangle \otimes |j\rangle = \sum_{i=0}^{1} |i\rangle \otimes G^i |j\rangle \rightarrow C^i NOT_j |i\rangle \otimes |j\rangle = \sum_{i=0}^{1} |i\rangle \otimes NOT^i |j\rangle \tag{5}$$

The 1-layer VQC working as the basis neuron in this model is called *Strongly Entangling Circuit* (SEC). It gives the idea to prepare quantum states generating a strong entanglement inside each circuit neuron. It prepares quantum states generating a strong entanglement using the $C^a NOT_b$ gates inside each circuit neuron. The SEC $U(W)$ is described in (6), then we define the following indexes and quantities: (a) n = number of wires in the circuit; (b) r = range for the imprimitives, range to select the control wire, (c) c_j are the nodes for the controls, $(jr - r)$ in total; (d) $t_j = jr$ are the nodes for the controlled qubits, jr in total; and (e) $W_j = (\phi_j, \theta_j, \omega_j)$, the array with the trainable parameters (for each wire in the circuit). Based on this, the dimensions of the total array of trainable parameters, $W = [W_1 W_2 \dots W_n]$, is $n \times 3$. Then:

$$U(W) = \prod_{k=1}^{n} C^{c_k} NOT_{t_k} \prod_{j=1}^{n} R(W_j) \tag{6}$$

2.4 Measurements and Loss Function

To generate this hybrid quantum-classical model there are needed certain interactions between both parts to build the whole model. The classical information was introduced in the Quantum Circuit after the angle embedding, and then the quantum states generated after embedding are processed in the circuit. In the QML model the output of the model can be taken directly from the circuit as a measurement in one of the states or certain measurements can be taken and processed afterward in a classical model so it ends up as a classical-quantum-classical model.

In the field of Quantum Information, a measurement is an operation made on a qubit, which is in superposition in the basis vectors, so a classical bit or classical result no longer in superposition is obtained. During the development of this research, we are not working with a mere measurement, but with the expectation value, which is the probabilistic expected value of a measurement on a qubit. So in the VQC to get the quantum data into classical data and continue with another stage of the model, there are needed to take the expectation values of different wires (qubits, states) in the circuit, which may be continuously vary depending on the rest of the circuit, particularly in the change of the trainable parameters during the training.

As mentioned before, the trainable parameters in the circuit are optimized during the training depending on two main elements: Optimization algorithm and Loss Function (Cost Function). The Loss Function takes the prediction-output $\hat{y}$ value of the whole circuit and calculates depending on the function the error with the expected value y. Depending on the model's task the Loss Function is selected, in this case as it will be explained in the following section, the selected task is sequence modeling for classification. Considering this fact, the selected Loss Function is called Binary Cross-entropy [15] which is used for models in which output is the probability that an input belongs to a particular class. The Loss function is presented in the following equation considering N training examples:

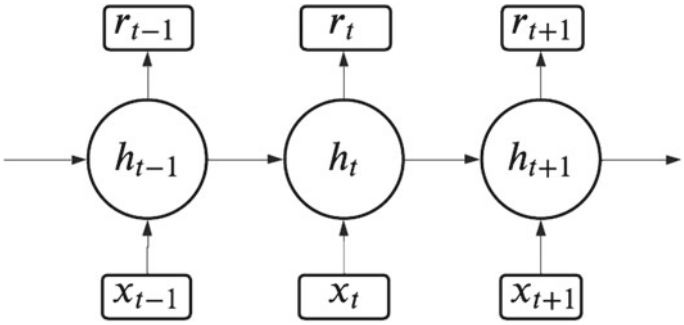


Fig. 3 Representation of some steps of the Recurrent Neural Network

$$L = -\frac{1}{N}\sum_{i=1}^{N} y_i \cdot \log(\hat{y}) + (1 - y_i) \cdot \log(1 - \hat{y}) \tag{7}$$

The Optimization algorithm, using the result of the Loss Function optimizes the weights or trainable parameters in the model by performing a modified version of a normal Gradient Descent, in which by back-propagating the error obtained by the Loss Function considering a differentiable model (function model), the trainable parameters get update till finding (ideally) a global minimum in which the values of the trainable parameters obtain the global minimum error. Specifically, we perform different tests using the following Optimization Algorithms: Stochastic Gradient Descent (SGD) [16] and Adam [17].

3 Strongly Entangling Neural Network

In the previous section, different elements were introduced and explained in detail to use them as construction blocks for the model presented in this section. The proposed model architecture has been called *Strongly Entangling Neural Network* (SENN). The Circuit Neuron presented in Fig. 2 is used as the basis for the creation of a model intending to imitate part of the behavior of an RNN. It is a type of neural network, where a closed loop between the units inside a layer generates a Forward Propagation among those hidden units. It causes the output of a unit can affect a subsequent one, then allowing a temporal dynamic behavior. The output of the unit $t - 1$ affects the unit t, whose output then affects unit $t + 1$ and so on, as illustrated in Fig. 3. RNNs are used for sequenced inputs with temporal behavior so that the behavior of previous inputs influences subsequent units.

The purpose is that SENN can be used for any kind of sequence modeling. However, the architecture will be explained for a specific use case: QNLP Classification Task. The model consists of 6 layers which can be either Classical Layers (CL) or

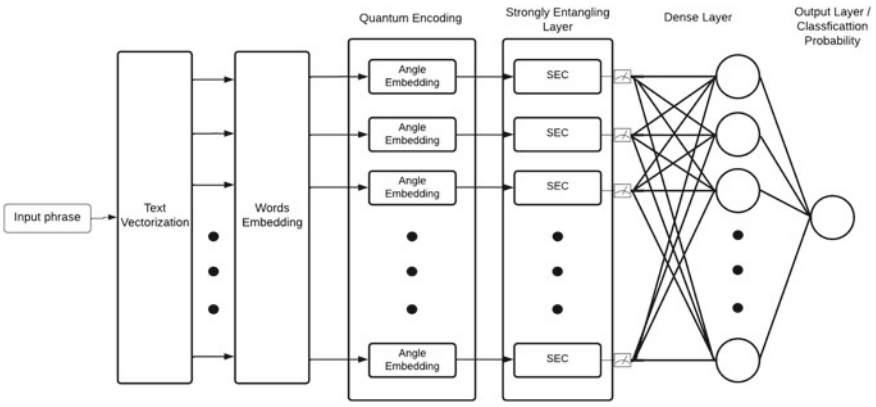


Fig. 4 Entire diagram for the Strongly Entangling Neural Network showing its several layers

Quantum Layers (QL): (1) TV Layer (CL), (2) Embedding Layer (CL), (3) Quantum Encoding/Angle Embedding (QL), (4) Strongly Entangling Layers (QL), (5) Dense Layer (CL), and (6) Output Dense Layer (CL). The whole model can be seen in Fig. 4. The TV layer is a preparing step for the Embedding layer which simply maps text features into integer sequences [18].

The data flow in the architecture is as follows: A phrase enters the TV Layer and gets converted into an integer sequence. It then enters in the Embedding Layer, where every word gets converted into a vector representation of size d (considering $d = 3$ in the experiment explained in the following section). Also, in Fig. 2, the same value for d was used. Considering n maximum words in each phrase entering into the circuit, then, the Embedding Layer outputs the same n number of vectors which now enters n Circuit Neurons as explained in the previous section (consisting of the Angle Embedding and the SEC as it can be seen in Fig. 2, a circuit neuron). The workflow generates each word converted into an embedding that will be processed by a Circuit Neuron. It was tried to generate a forced Forward Propagation between circuit neurons using some two-qubit entangling gates such as $C^a NOT_b$ gates, but it was found that this decreased the accuracy of the model and increased the loss. Decisions about this point are discussed in the following section. Considering the length of the embedding, then: $Total\ number\ of\ qubits = embedding\ length \times words\ per\ phrase.$ To extract the information from the quantum circuits (n circuit neurons), it is needed to measure the qubits according to the explanation given in the next section. There, taking as mentioned, the expectation value using a Z Pauli Gate.

The measurements enter into a Dense Layer with a number of hidden units equal to the number of qubits using a *ReLU* (rectified linear unit) activation function [19]. Finally, the last layer is a Dense Layer with a single hidden neuron using *Sigmoid* activation function [19] considering a classification task having then a stochastic output. The whole architecture is by itself very similar to some other NLP architectures, just replacing the RNN, a Long Short-Term Memory layer for example, with the QE

and the Strongly Entangling Layer, getting the model completely into the field of QML and giving the possibility of treat sequential data in the form of NLD in the context of Quantum Computing.

For the experimentation, a dataset provided by [20] was used, which contains phrases translated from English to Spanish. This dataset was originally used for Deep Learning models with translation tasks, but it was modified for its use in the model test by generating a labeled dataset of phrases in both, English and Spanish, having a $1-$label for phrases in English and a $0-$label for phrases in Spanish. With this dataset in mind, the experiment consisted of using the model to create a text sorter for English and Spanish. The dataset has more than $230,000$ training examples but considering the computational cost for the Proof of Concept, a reduced version from the original dataset was used. The model was trained for just 25 epochs, to see the performance and evolution considering a small training time. The number of epochs is defined as the number of times that the model sees the whole training dataset, which means that is the number of times that the model iterates and optimizes the trainable variables over the dataset.

All the programming for the model and experimentation was done in Python environment using two main libraries: (a) Tensorflow [18] to generate the model and for the training; and (b) PennyLane [21] for the generation of all the Quantum layers. PennyLane has integrated a tool allowing to convert a circuit into a Tensorflow Layer, letting the integration of the Classical Layers and the Quantum Layers into a single model. PennyLane has different devices available for the simulation of quantum circuits. In this case, the configuration *Default.Qubit* [21] was used. The results of the experimentation performed are shown and discussed in the following section.

4 Results

The experiments were done considering the details explained in the last section. After 25 epochs the model achieved a Loss of 0.0768, Validation Loss of 0.1641, Accuracy of 0.9770, and Validation Accuracy of 0.9100. The validation of the model is an extra step in each epoch in which a Validation Dataset different from the training dataset is passed to the model to evaluate how the model performs with data that it has never seen before. Weights are not optimized during validation. The Losses and Accuracy changes during training can be seen in Fig. 5.

To get these results different decisions were taken related to the number of qubits used, embedding length, and used gates in circuits, among other parameters of the model. As a summary of the model, in this case, the following values were selected for all these parameters: (a) Embedding Length: 3, (b) Imprimitive 2-qubit gate: C^aNOT_b, (c) Maximum length for the phrases: 4 words, (d) Pauli gate used for the expectation values: Z, and using the previous values we can calculate (e) $Total\ number\ of\ qubits = embedding\,length \times words\ per\ phrase = 12\,qubits$. As it can be seen in Fig. 5, Loss and Validation loss decrease properly in every epoch. Similarly, Accuracy and Validation Accuracy increases properly in

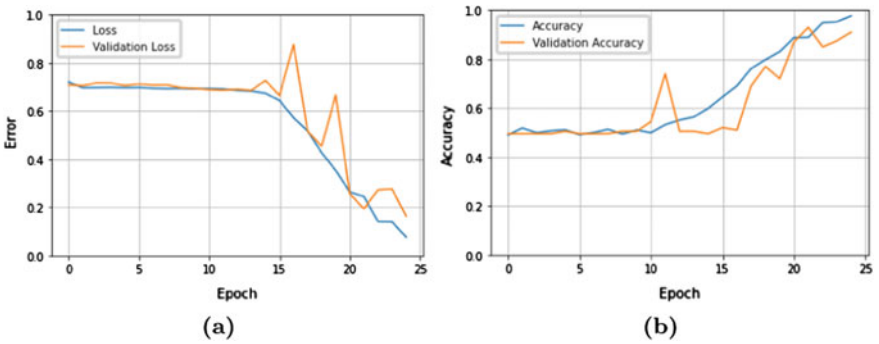


Fig. 5 **a** Loss and Validation Loss during training. **b** Accuracy and Validation Accuracy during training

every epoch. As can be appreciated in the same figure, there are no major changes in Loss or in Accuracy during the first 10 epochs. This is since the fact that in the model the weights in the Quantum circuits were initialized randomly, which correspond to the rotation angles in the circuits, so the model needs to learn completely by itself the ranges of the values for these angles. But, once that finds the proper pattern, it starts to improve rapidly, increasing the accuracy to around 0.47 in less than 15 epochs.

As mentioned before and as it can be seen in Fig. 4, the model still does not use an explicit Forward propagation between the circuit neurons, but instead takes the expectation value in every qubit and uses an extra dense layer, where connections between expectation values of different qubits are inferred by the model during training. This was expected to affect the model performance, but that is not the case as can be seen in the results, having then a quantum-classical hybrid model that is able to encode text and learn from it to perform a specific task. This forward propagation is needed to generate more complex models based on this one. Details about this point are discussed in the following section where conclusions and further work are mentioned.

5 Conclusions

During research and experimentation, we were able to encode NLD in a Quantum-Classical hybrid model, which can learn from the data and perform properly a classification task. At first instance, it was desired to have a model analog to an RNN with an explicit forward propagation using $C^a NOT_b$ gates between circuit neurons. But during experimentation, it was found that since the circuit neurons use a Strongly Entangling Circuit with $C^a NOT_b$ gates, the use of extra $C^a NOT_b$ between circuit neurons will generate an entanglement of all the qubits in the model, then complicating more the learning task and affecting the performance. Due to these reasons, it was discarded to use a 2-qubit entangling gate to generate this forward propagation. It was chosen to leave the model without this step since it got proper results

in this simpler version. Despite this, it is left to future research to find some mechanism to generate forward propagation. In fact, it is possible to use this base to build more complex models, for example applying SENN as a base to generate a Quantum Attention model or a Quantum Transformer model.

For this research, the focus has been put almost uniquely on QNLP, leaving behind other Sequence Modelling problems. In future work, it is planned to implement SENN with different kinds of sequence data to get a complete generalization. For the current proposal, the adequate use of quantum circuits and the proper interactions between quantum and classical parts have allowed taking advantage by exploiting the features of quantum processing reaching a high accuracy in just a few epochs. The classification task solved for the experimentation can be considered simple, but it still can be grown to more complex problems and architectures in different fields of SM and particularly in QNLP.

Acknowledgements J. Ismael Díaz-Ortiz acknowledges Emmanuel Díaz the unconditional support given at every time. The authors would like to acknowledge the financial support of NOVUS (Grant number: 214), Institute for the Future of Education, Tecnologico de Monterrey, Mexico, in the production of this work.

References

1. Biamonte, J., Wittek, P., Pancotti, N., Rebentrost, P., Wiebe, N., Lloyd, S.: Quantum machine learning. Nature **549**(7671), 195–202 (2017)
2. Yousuf, H., Lahzi, M., Salloum, S.A., Shaalan, K.: A systematic review on sequence-to-sequence learning with neural network and its models. Int. J. Elec. Comp. Eng. **11**(3), 2315 (2021)
3. Manning, C.D., Schutze, H.: Foundations of Statistical Natural Language Processing (Chap. 16), pp. 575–608. MIT Press, England (1999)
4. Koshorek, O., Cohen, A., Mor, N., Rotman, M., Berant, J.: Text segmentation as a supervised learning task. In: Proceedings of the 2018 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, vol. 2 (Short Papers), pp. 469–473. Association for Computational Linguistics, New Orleans, Louisiana (2018)
5. Tjong Kim Sang, E.F., De Meulder, F.: Introduction to the CoNLL-2003 shared task: Language-independent named entity recognition. In: Proceedings of Seventh Conference on Natural Language Learning at HLT-NAACL 2003, pp. 142–147 (2003)
6. Goodfellow, I., Bengio, Y., Courville, A.: Deep Learning (Chaps. 9–10), pp. 326–415. MIT Press (2016)
7. Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A.N., Kaiser, L., Polosukhin, I.: Attention is all you need (2017)
8. Tunstall, L., von Werra, L., Wolf, T.: Natural Language Processing with Transformers: Building Language Applications with Hugging Face. O'Reilly Media Inc. (2022)
9. Brown, T.B., Mann, B., Ryder, N., Subbiah, M., Kaplan, J., Dhariwal, P., Neelakantan, A., Shyam, P., Sastry, G., Askell, A., Agarwal, S., Herbert-Voss, A., Krueger, G., Henighan, T., Child, R., Ramesh, A., Ziegler, D.M., Wu, J., Winter, C., Hesse, C., Chen, M., Sigler, E., Litwin, M., Gray, S., Chess, B., Clark, J., Berner, C., McCandlish, S., Radford, A., Sutskever, I., Amodei, D.: Language models are few-shot learners (2020)

10. Devlin, J., Chang, M.W., Lee, K., Toutanova, K.: BERT: Pre-training of deep bidirectional transformers for language understanding. In: Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, vol. 1 (Long and Short Papers), pp. 4171–4186. Association for Computational Linguistics, Minneapolis, Minnesota (2019)
11. Mitchell, M., Pistilli, G., Jernite, Y., Ozoani, E., Gerchick, M., Rajani, N., Luccioni, S., Solaiman, I., Masoud, M., Nikpoor, S., Ferrandis, C.M., Bekman, S., Akiki, C., Contractor, D., Lansky, D., McMillan-Major, A., Thrush, T., Ilić, S., Dupont, G., Longpre, S., Dey, M., Biderman, S., Kiela, D., Baylor, E., Scao, T.L., Gokaslan, A., Launay, J., Muennighoff, N.: Large open-science open-access multilingual language model (2022). https://huggingface.co/bigscience/bloom
12. Schuld, M., Bocharov, A., Svore, K.M., Wiebe, N.: Circuit-centric quantum classifiers. Phys. Rev. A **101**(3) (2020)
13. Vajjala, S., Majumder, B., Gupta, A., Surana, H.: Practical Natural Language Processing. O'Reilly Media, Sebastopol, CA (2020)
14. Schuld, M., Petruccione, F.: Supervised Learning with Quantum Computers. Springer International Publishing (2018)
15. Ruby, U., Yendapalli, V.: Binary cross entropy with deep learning technique for image classification. Int. J. Adv. Trends Comput. Sci. Eng. **9**(10) (2020)
16. Ruder, S.: An overview of gradient descent optimization algorithms (2017)
17. Kingma, D.P., Ba, J.: Adam: a method for stochastic optimization (2017)
18. Abadi, M., Agarwal, A., Barham, P., Brevdo, E., Chen, Z., Citro, C., Corrado, G.S., Davis, A., Dean, J., Devin, M., Ghemawat, S., Goodfellow, I., Harp, A., Irving, G., Isard, M., Jia, Y., Jozefowicz, R., Kaiser, L., Kudlur, M., Levenberg, J., Mané, D., Monga, R., Moore, S., Murray, D., Olah, C., Schuster, M., Shlens, J., Steiner, B., Sutskever, I., Talwar, K., Tucker, P., Vanhoucke, V., Vasudevan, V., Viégas, F., Vinyals, O., Warden, P., Wattenberg, M., Wicke, M., Yu, Y., Zheng, X.: TensorFlow: Large-scale machine learning on heterogeneous systems (2015). https://www.tensorflow.org/
19. Nwankpa, C., Ijomah, W., Gachagan, A., Marshall, S.: Activation functions: comparison of trends in practice and research for deep learning (2018)
20. Kelly, C.: Tab-delimited bilingual sentence pairs (2022). https://www.manythings.org/anki/
21. Bergholm, V., Izaac, J., Schuld, M., Gogolin, C., Ahmed, S., Ajith, V., Alam, M.S., Alonso-Linaje, G., AkashNarayanan, B., Asadi, A., Arrazola, J.M., Azad, U., Banning, S., Blank, C., Bromley, T.R., Cordier, B.A., Ceroni, J., Delgado, A., Matteo, O.D., Dusko, A., Garg, T., Guala, D., Hayes, A., Hill, R., Ijaz, A., Isacsson, T., Ittah, D., Jahangiri, S., Jain, P., Jiang, E., Khandelwal, A., Kottmann, K., Lang, R.A., Lee, C., Loke, T., Lowe, A., McKiernan, K., Meyer, J.J., Montañez-Barrera, J.A., Moyard, R., Niu, Z., O'Riordan, L.J., Oud, S., Panigrahi, A., Park, C.Y., Polatajko, D., Quesada, N., Roberts, C., Sá, N., Schoch, I., Shi, B., Shu, S., Sim, S., Singh, A., Strandberg, I., Soni, J., Száva, A., Thabet, S., Vargas-Hernández, R.A., Vincent, T., Vitucci, N., Weber, M., Wierichs, D., Wiersema, R., Willmann, M., Wong, V., Zhang, S., Killoran, N.: Pennylane: automatic differentiation of hybrid quantum-classical computations (2022)